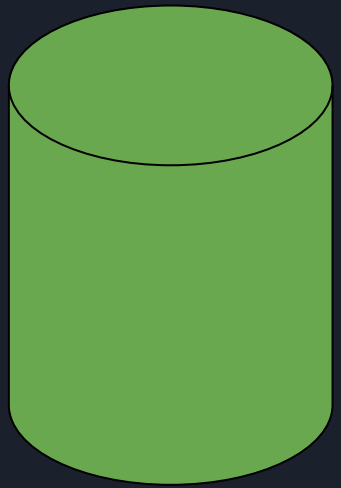
GESTIÓN DE DATOS

CONCURRENCIA

RECORDERIS...



¿QUÉ APRENDEREMOS HOY?



- Concepto de "concurrencia"
- Problemas de mal manejo de concurrencia
- Niveles de aislamiento
- Bloqueo y desbloqueo de recursos

CONCEPTO: Transacción



CONCEPTO: Concurrencia

Definiciones hay muchísimas!

Todo se resume a:

"Qué puede salir mal si dos usuarios/procesos/hilos tocan lo mismo al mismo tiempo"

Aplica para la vida cotidiana, no solo a sistemas!

CONCEPTO: Concurrencia

¿Existe algún inconveniente si varios usuarios realizan una operación simultáneamente sobre un mismo campo en una tabla de una base de datos?

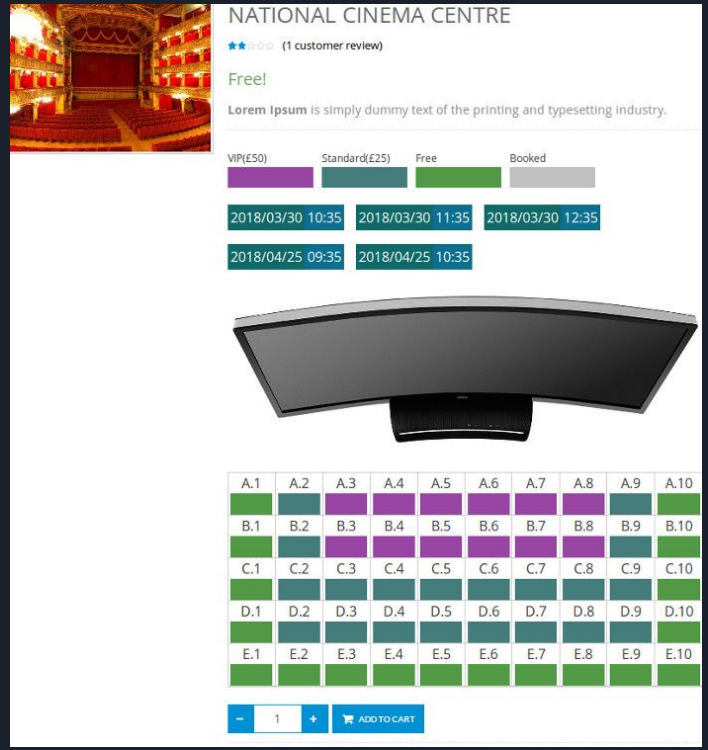
¿Y sobre un archivo?

Ejemplo Ilustrativo (1 / 3)

Leidy hace una reserva en un sistema en línea para ir a cine a ver su película favorita. Busca el teatro XYZ y escoge la función de las 3:00 pm, en el asiento K11.

Al mismo tiempo, Diana (que no se conoce con Leidy pero casualmente tienen gustos parecidos) realiza la reserva para la misma película, en el mismo teatro, misma hora y mismo asiento.

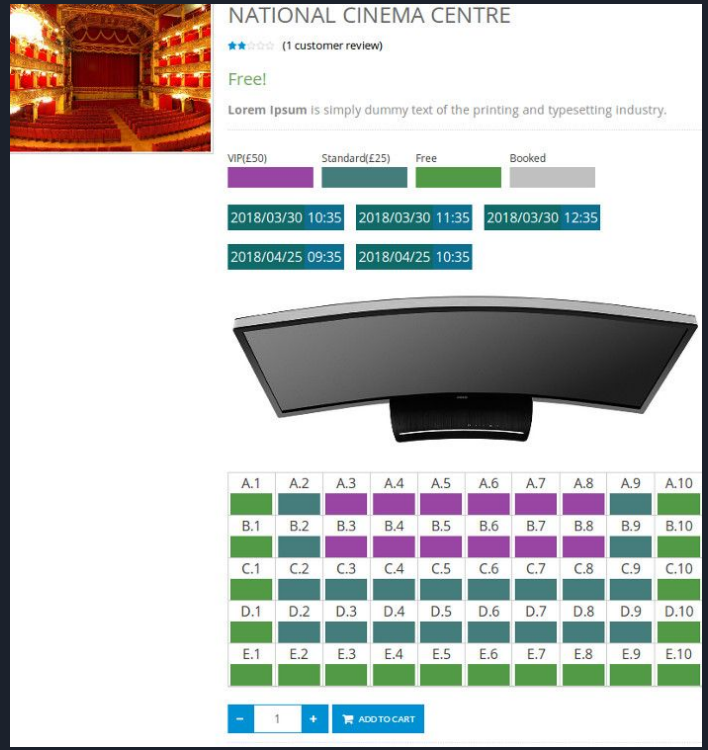
Leidy confirma la reserva y hace el pago. Al cabo de 2 segundos, Diana también confirma la reserva y hace el pago.



Ejemplo Ilustrativo (2 / 3)

PREGUNTA

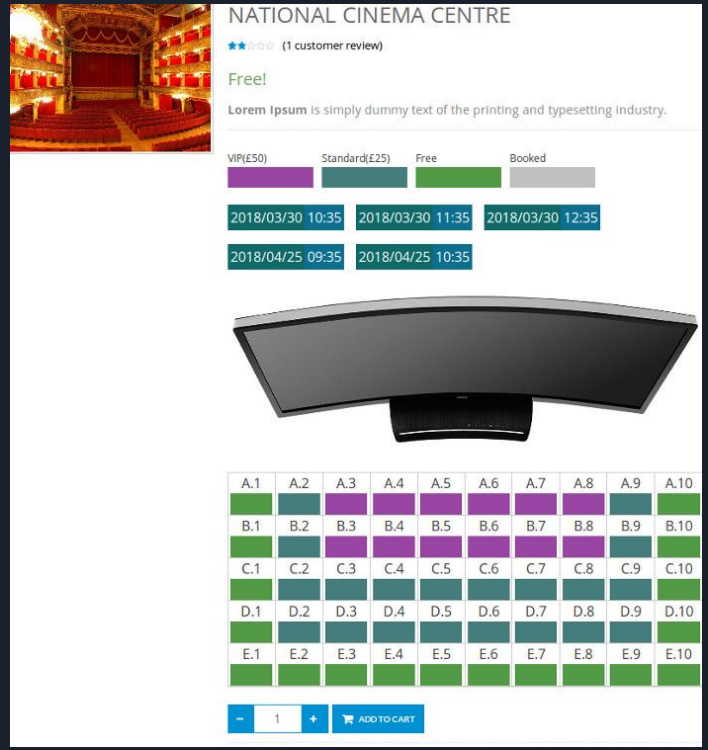
Asumiendo que el caso anterior trabaja con un sistema monousuario, entonces ¿Finalmente a nombre de quien quedó el asiento reservado?



Ejemplo Ilustrativo (3 / 3)

```
-- sesión de Leidy
update   seats
set      status = 'RESERVED'
where    theater = 'XYZ'
and      seat_id = 'K11'
and      movie_name = 'some_movie'
and      movie_time = '15:00:00'
and      user_id = 'Leidy'


-- sesión de Diana (2 segundos después)
update   seats
set      status = 'RESERVED'
where    theater = 'XYZ'
and      seat_id = 'K11'
and      movie_name = 'some_movie'
and      movie_time = '15:00:00'
and      user_id = 'Diana'
```



PREGUNTA!!!

¿El problema de los asientos pudo evitarse? ¿Cómo y porqué si o porqué no?

(discusión verbal)

EJEMPLO: Código fuente

```php
// concurrence.php (ejecutar en varios procesos)

<?php
    $dataFile = '/tmp/value.txt';
    $x = file_get_contents($dataFile);  // 40
    $x = $x + 10;

    sleep(rand(1,4));  // Simular operación externa lenta

    file_put_contents($dataFile, $x);    // ??
?>
```

EJEMPLO: Código fuente

```php
// concurrence.php

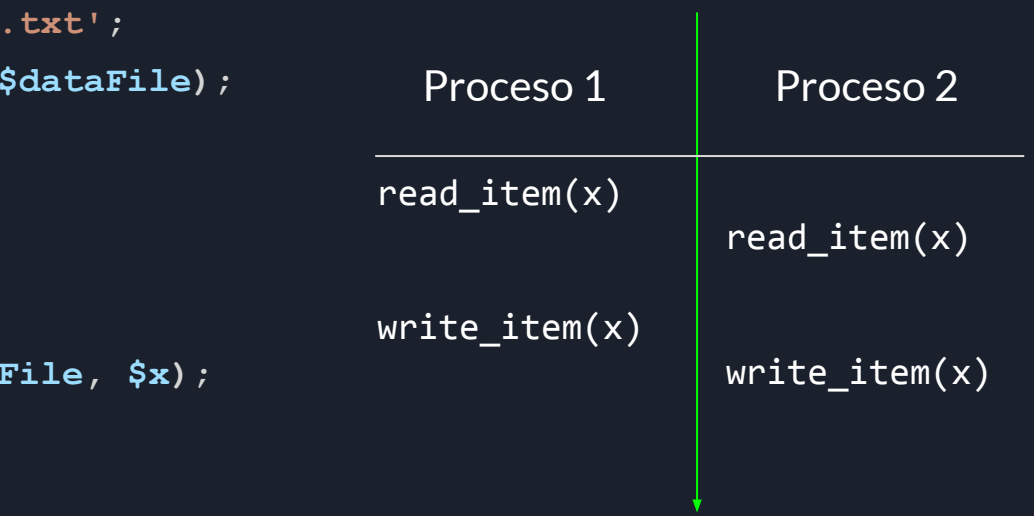
<?php
    $dataFile = '/tmp/value.txt';
    $x = file_get_contents($dataFile);
    $x = $x + 10;

    sleep(rand(1,4));

    file_put_contents($dataFile, $x);
?>
```

¿En qué valor queda $x ?

CASO 1:



EJEMPLO: Código fuente

```php
// concurrence.php

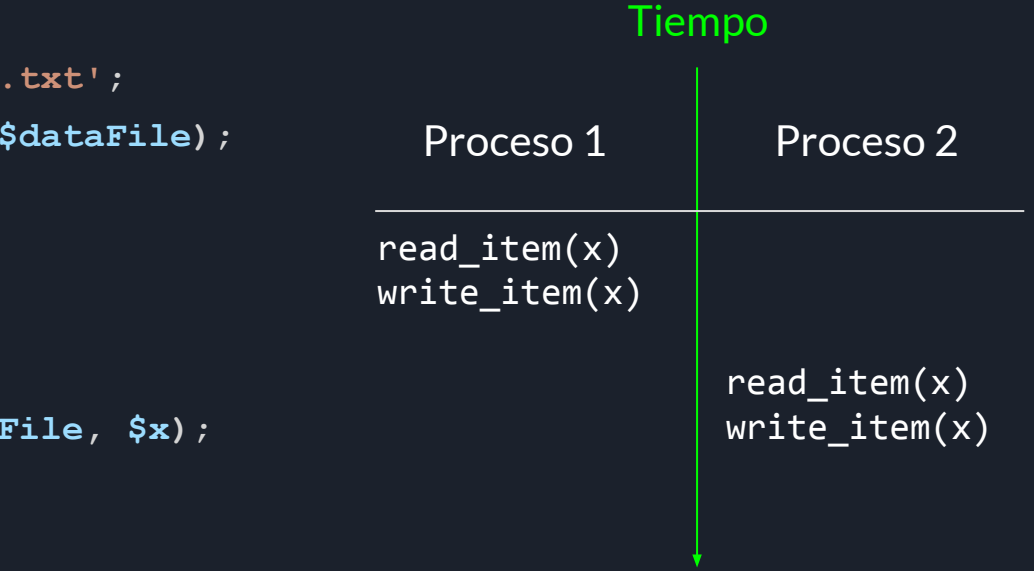
<?php
    $dataFile = '/tmp/value.txt';
    $x = file_get_contents($dataFile);
    $x = $x + 10;

    sleep(rand(1,4));

    file_put_contents($dataFile, $x);
?>
```

¿En qué valor queda $x ?

CASO 2:



EJEMPLO: Código fuente

```php
// concurrence.php

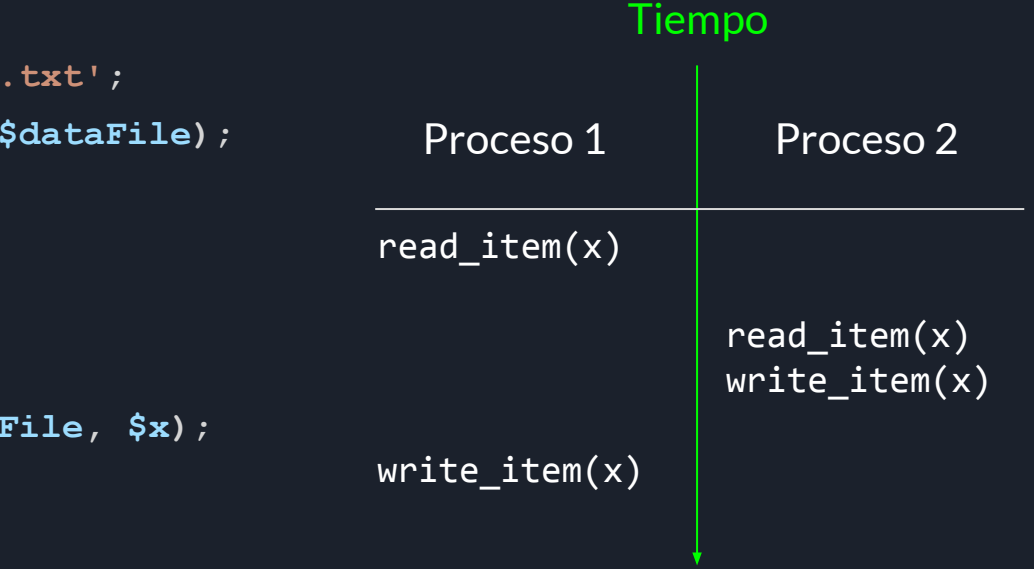
<?php
    $dataFile = '/tmp/value.txt';
    $x = file_get_contents($dataFile);
    $x = $x + 10;

    sleep(rand(1,4));

    file_put_contents($dataFile, $x);
?>
```

¿En qué valor queda $x ?

CASO 3:



PREGUNTA!!!

Y finalmente… ¿Qué es *concurrencia*?

RETOMANDO...

¿Existe algún inconveniente si varios usuarios realizan una operación simultáneamente sobre un mismo campo en una tabla de una base de datos?

Respuesta: Si. Y esto se soluciona con el "control de concurrencia" y el "nivel de aislamiento".

CONCEPTO: Concurrencia

Ejemplos

- Archivo de office en una "nas", abierto por varios usuarios
- Sistema de reservas en línea
- Una fila en una cafetería
- Una llamada telefónica (unidirecccional o bidireccional)

Concurrencia:
Problemas de aislamiento (cuando los datos mienten)

Def: Lectura sucia (dirty read)

Se tienen dos procesos: pr1 y pr2

- pr1 lee los asientos disponibles y entre ellos está el K11 en estado "libre"
- pr1 inicia una transacción
- pr1 modifica el estado del asiento K11 a "reservado"
- pr2 lee los asientos disponibles y entre ellos está el K11 "reservado" (no lo puede reservar, ya ocupado)
- pr1 deshace la transacción (rollback)



🔰 El dato que pr2 tiene ya no es consistente frente al momento inicial

➡️ pr2 tomó decisiones sobre un dato que nunca existió

Def: Actualización perdida (lost update)

Se tienen dos procesos: pr1 y pr2

- pr1 lee el asiento K11 en estado "libre"
- pr2 lee el asiento K11 en estado "libre"
- pr1 inicia una transacción
- pr2 inicia una transacción
- pr1 modifica el estado del asiento K11 a "reservado"
- pr2 modifica el estado del asiento K11 a "comprado"
- pr1 confirma la transacción (commit)
- pr2 confirma la transacción (commit)



🔰 El dato que pr1 escribió se perdió para siempre

➡️ pr2 modificó un dato sin saber que pr1 ya lo estaba cambiando

Def: Lectura no repetible (non repeatable read)

Se tienen dos procesos: pr1 y pr2



- pr1 lee el asiento K11 en estado "libre"
- pr2 lee el asiento K11 en estado "libre"
- pr2 inicia una transacción
- pr2 modifica el estado del asiento K11 a "comprado"
- pr2 confirma la transacción (commit)
- pr1 reintenta la lectura del asiento K11 y ya aparece en estado "comprado"

🔰 ***El dato*** que pr1 leyó cambió en segundos o milisegundos

➡️ pr1 obtuvo 2 respuestas diferentes para una misma consulta

Def: Lectura fantasma (phantom read)

Se tienen dos procesos: pr1 y pr2

- pr1 lee el detalle de los asientos libres y encuentra 100 registros
- pr2 inicia una transacción
- pr2 inserta un nuevo asiento (K99)
- pr2 confirma la transacción (commit)
- pr1 reintenta la lectura del detalle de los asientos libres y ya hay 101



🔰 ***La cantidad de registros*** que pr1 leyó cambió en segundos o milisegundos

➡️ No hubo cambio en los datos, sino en la cantidad de registros

PREGUNTA PARA ANALIZAR

El proceso 1 está realizando una operación de ***agregación*** sobre N registros.

El proceso 2 está actualizando uno de esos registros.

¿Es posible que, al finalizar ambos procesos, los resultados sean no confiables?

PREGUNTA PARA ANALIZAR

¿Por qué pueden presentarse los problemas de concurrencia?

¿Qué técnicas conoce usted para manejar estos problemas?

¿Qué otras alternativas sugiere usted para mitigar los inconvenientes del mal manejo de la concurrencia?

Tabla comparativa

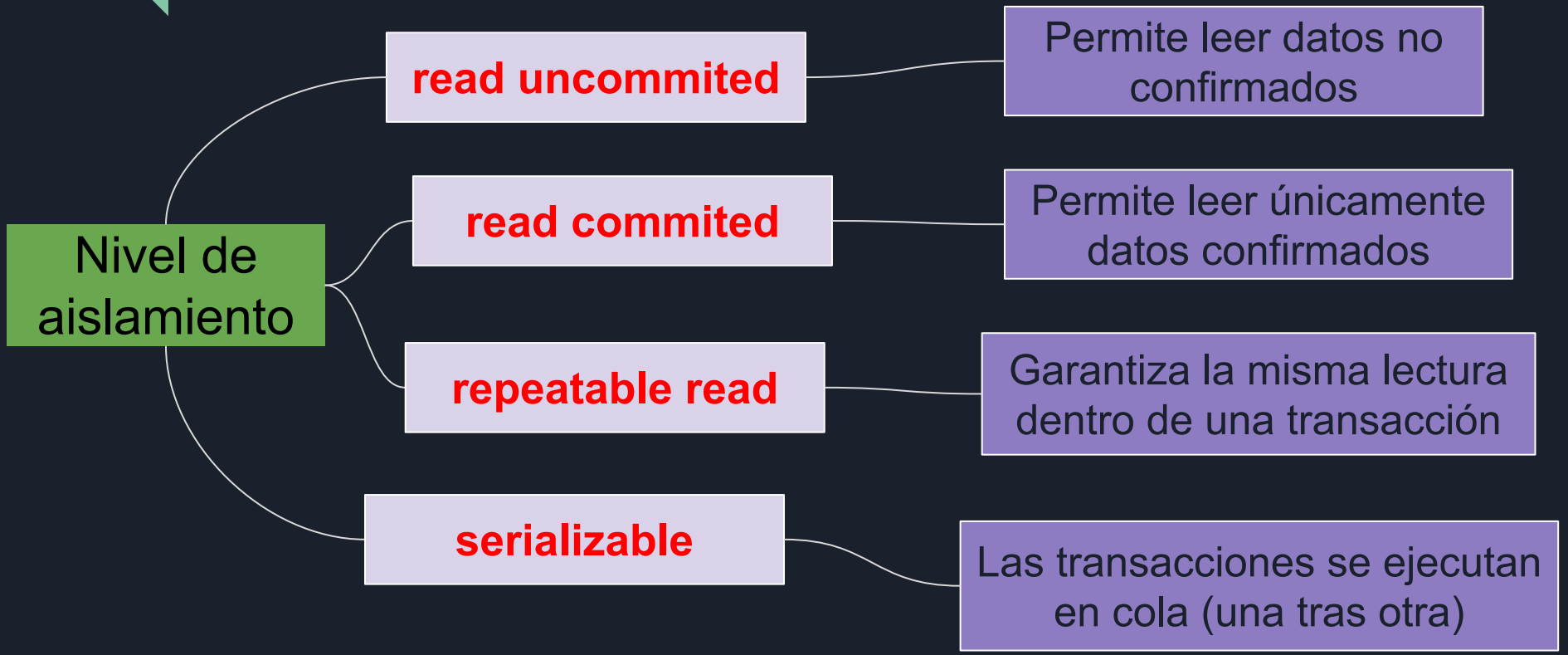
	dirty read	lost update	phantom read	non repeatable read
definición	Leer algo que aún no ha sido confirmado	Dos transacciones leen el mismo valor y una lo sobreescribe sin tener en cuenta a la otra	Cambia el conjunto de registros con la misma condición	Leer el mismo dato 2 veces con resultados diferentes
causa	Otra transacción modificó datos y no confirmó	Otra transacción modificó un mismo dato sin verificar si ya estaba siendo cambiado	Otra transacción modificó la cantidad de registros	Otra transacción modificó datos entre 2 lecturas o más
conse-cuencias	Tomar decisiones con datos que no existieron	Datos inconsistentes	Estadísticas incompletas	Datos inconsistentes

Concurrencia:
Niveles de aislamiento

Concepto: Nivel de aislamiento

Indica que tan protegida está una transacción de efectos de otras transacciones.

Concepto: Niveles de aislamiento



Concepto: Niveles de aislamiento

Nivel de aislamiento	Evita Dirty Read	Evita Non Repeatable Read	Evita Phantom Read	Evita Lost Update *	Observaciones
Read Uncommitted	❌	❌	❌	❌	⬆️ Velocidad ⬇️ Consistencia
Read Committed *	✅	❌	❌	⚠️	⬆️ Velocidad ↔ Consistencia
Repeatable Read **	✅	✅	⚠️ **	⚠️	↔ Velocidad ↔ Consistencia
Serializable ***	✅	✅	✅	⚠️	⬇️ Velocidad ⬆️ Consistencia

* nivel por defecto en postgresql, oracle, sqlserver
** nivel por defecto en mariadb/mysql
*** nivel por defecto en sqlite

* depende del dbms y del backend
** no es solo el dbms, también depende del diseño del backend

¿Cuando usar cual?

read uncommited	Pruebas Aplicaciones monousuario
read commited	Tiendas en línea pequeñas Aplicaciones con poco movimiento
repeatable read	Aplicaciones con cálculos repetitivos en un mismo conjunto
serializable	Sistemas financieros Control de inventario

Concepto: Nivel de aislamiento

IMPORTANTE:

Mayor aislamiento ⇒ Mayor consistencia

Mayor consistencia ⇒ Menor concurrencia

PREGUNTA PARA ANALIZAR

¿Usted considera que alguno de los siguientes tipos de fallos podrían originarse por mal manejo de concurrencia?

- Problemas de hardware
- Caídas de sistema
- Errores de escritura en disco
- Lecturas con datos incorrectos
- Acceso no autorizado
- Exposición de datos sensibles
- Errores de autenticación
- Errores de autorización

PREGUNTA PARA ANALIZAR

¿Usted considera que alguno de los siguientes tipos de fallos podrían ser una consecuencia del mal manejo de concurrencia?

- Problemas de hardware
- Caídas de sistema
- Errores de escritura en disco
- Lecturas con datos incorrectos
- Acceso no autorizado
- Exposición de datos sensibles
- Errores de autenticación
- Errores de autorización

Concurrencia en mariadb/mysql

lock tables

Bloquea una tabla frente a operaciones de escritura por parte de otra sesión (otro usuario).

Implícitamente también bloquea las claves foráneas relacionadas a esta tabla

unlock tables

Finaliza un bloqueo iniciado por "lock tables"

Concurrencia en mariadb/mysql

lock tables read

Bloquea una tabla de forma que ninguna sesión (ni siquiera la actual) pueda actualizarla hasta que se desbloquee, aunque sí se puede leer.

lock tables write

Permite que únicamente la sesión actual pueda actualizar la tabla. Las otras sesiones no podrán leer ni escribir mientras la tabla esté bloqueada

Transacciones en mariadb/mysql

IMPORTANTE!!!

Repite conmigo:

"Siempre debo desbloquear las tablas con unlock después de haberlas bloqueado con lock"

Concurrencia - ejemplo

Ejercicio 1: Analizar la siguiente situación

Sesión 1		Sesión 2	
`lock table people write;`			
`select * from people;`			
		`select * from people;`	
`insert into people values (3, 'SHAKIRA');`			
`unlock tables;`			

Concurrencia - ejemplo

Ejercicio 1: Analizar la siguiente situación

Sesión 1		Sesión 2	
`lock table people write;`			
`select * from people;`			
		`select * from people;`	
`insert into people values (3, 'SHAKIRA');`			
`unlock tables;`			
			

Concurrencia - ejemplo

Ejercicio 2: Analizar la siguiente situación

Sesión 1		Sesión 2	
`lock table people read;`			
`select * from people;`			
		`select * from people;`	
`insert into people values (3, 'SHAKIRA');`			
`unlock tables;`			

Concurrencia - ejemplo

Ejercicio 2: Analizar la siguiente situación

Sesión 1		Sesión 2	
`lock table people read;`			
`select * from people;`			
		`select * from people;`	
`insert into people values (3, 'SHAKIRA');`			
`unlock tables;`			

Concurrencia:
Problemas de bloqueos (cuando las transacciones se odian mutuamente)

Def: Abrazo mortal (deadlock)

Ejemplo ilustrativo



Amor, cuelga tú primero

No, yo quiero que cuelgues tú

Acá me quedaré hasta que cuelgues

No, dale primero tú!

No, dale tú primero

Y yo también me voy a quedar



Def: Abrazo mortal (deadlock)

... también llamado "interbloqueo".

Se tienen dos procesos: pr1 y pr2

- pr1 bloquea el registro A1
- pr2 bloquea el registro B2

- pr1 actualiza el registro B2
- pr2 actualiza el registro A1



🔰 Ambos procesos quedan bloqueados mutuamente

➡️ Como ningún proceso dará el *brazo a torcer*, el dbms decide cual transacción finalizar.

Def: Inanición (starvation)

Se tienen varios procesos: pr1, pr2, pr3, pr4, pr5...

- pr1 decide que debe bloquear el registro A1
- pr2 solicita bloqueo del registro A1 (queda bloqueado)
- pr3 solicita bloqueo del registro A1 (queda en espera)
- pr4 solicita bloqueo del registro A1 (queda en espera)
- pr5 solicita bloqueo del registro A1 (queda en espera)
- ...
- ...



🔰 pr1 queda en una fila "eterna"

➡️ Nadie lo bloquea, pero tampoco tiene oportunidad de ejecutarse

Def: Sesgo de escritura (write skew)

Suponer que el usuario final pide que solo uno de ambos (A1, B2) debe estar activo

Se tienen dos procesos: pr1 y pr2



- pr1 lee A1 y B2 y ambos están activos
- pr2 lee A1 y B2 y ambos están activos

- pr1 desactiva A1 y confirma (commit)
- pr2 desactiva B2 y confirma (commit)

🔰 Ambos registros quedaron desactivados

➡️ El sistema va contra lo solicitado por el usuario

TABLA COMPARATIVA

	deadlock	starvation	Write skew
definición	Ninguno de dos procesos avanza	Un proceso no avanza	Ambos avanzan dejando inconsistencias
causa	Dependencias circulares	Mala asignación de prioridades	Decisiones basadas en cambios muy concurrentes
conse-cuencias	El dbms anula una transacción	Posibilidad de un tiempo de espera indefinido	Datos inconsistentes

Muchas de estas situaciones también dependen del backend, no solo del DBMS

PARA ANALIZAR...

Preguntas típicas

¿Coherencia o disponibilidad? ➡️ acid, base

¿Concurrencia o velocidad? ➡️ nivel de aislamiento

¿Debo mitigar problemas de concurrencia?

¿Debo mitigar problemas de bloqueos?



EJERCICIO: Abrazo mortal

Escribir código sql en MariaDB que dé como resultado un abrazo mortal.

EJERCICIO: Nivel de aislamiento

Investigar cómo configurar y cambiar el nivel de aislamiento en MariaDB

¿PREGUNTAS?

